

```

struct _bitfield {
    flagA : 1;
    flagB : 1;
    nybbA : 4;
    byteA : 8;
}

```

The numbers in the structure above are the field's size in bit's and not bytes. Here both flagA and flagB are 1-bit long so they're boolean variables. nybbA can store at most 4 bits so it can take a value to 0xFF (15 in Hexadecimal)

Fields in a bitfield can be address like a regular structure

```

struct _bitfield field;
field.flagA = 1;
field.flagB = 0;
field.nybbA = 0x0A;
field.byteA = 255;

```

Const

When a variable is declared as const, it means that the value of the variable will not be modified by the function. In embedded system paradigm declaring a variable to be const makes sure that the data in the variable is stored inside the ROM. An example for the first use case would look something like this:

```

void print_string( char const * the_string );

```

This makes it clear that the function will not modify the original string (note that we're passing the variable by reference here). An example for the second use case would be

```

const char * months[] = {
    "January", "February", "March",
    "April", "May", "June",
    "July", "August", "September",
    "October", "November", "December",
};

```